

System Requirements Documentation

Rachel Watts

Medical Esthetics Appointment & Client Management System

1. Customer Problem Statement

Medical esthetics clinics often struggle with managing appointments, provider availability, and service assignments efficiently. Scheduling errors can occur when providers are double-booked or not certified for a service. This system solves these issues by automating appointment scheduling and validation.

2. Functional Requirements

The system must:

- Store client, provider, and service data
- Allow scheduling of appointments
- Validate provider certification for services
- Check provider availability
- Display appointment details
- Prevent invalid appointments

3. System Design & Architecture

The system is a Java-based console application using object-oriented programming.

Components:

- Client class
- Provider class
- Service class
- Appointment class
- AppointmentScheduler class

4. Data Model Implementation

The system uses a structured data model based on relational database design concepts. Key entities include:

- Client
- Provider
- Service
- Appointment

Relationships:

- Clients can have multiple appointments
- Providers must be certified for services
- Appointments connect clients, providers, and services

5. User Interface Design

The system uses a console-based interface that:

- Displays available data
- Allows appointment scheduling
- Provides error messages for invalid inputs

6. Implementation Plan

The system was developed in phases:

1. Design database model
2. Create Java classes
3. Implement scheduling logic
4. Add validation and error handling
5. Test system functionality

7. Testing Design

Testing includes:

- Unit testing (individual functions)
- Integration testing (full scheduling process)
- System testing (complete workflow)

8. Error Handling

The system includes:

- Input validation
- Try-catch error handling
- User-friendly error messages

9. Collaboration & Code Integration

- GitHub used for version control
- Organized folder structure
- Separate testing modules

10. Performance Optimization

- Efficient object usage
- Minimal redundant operations
- Simple logic for fast execution

11. Maintenance & Support

- Easy to update classes
- Scalable for future features

- Code is readable and maintainable